

1. What is a Java Bean? Explain its main features and advantages

A **Java Bean** is a reusable Java class that follows specific conventions and is used to encapsulate multiple objects into a single object. It is mainly used in Java-based web applications (like JSP and Servlets) to represent data.

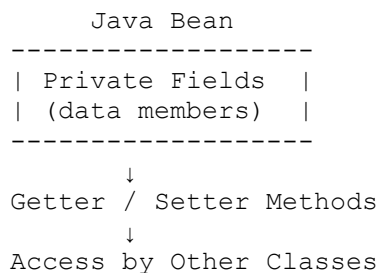
Java Beans are designed to make applications **modular, reusable, and easy to manage**. They help in separating **business logic from presentation logic**, which is an important concept in software design (MVC architecture).

A Java Bean acts like a **container of data** with controlled access using methods.

Main Features of Java Bean

1. **Private Variables (Encapsulation)**
 - All properties (variables) must be declared as `private`.
 - This ensures data security.
2. **Public Getter and Setter Methods**
 - Used to access and modify private variables.
 - Naming convention:
 - `getPropertyName()`
 - `setPropertyName()`
3. **No-Argument Constructor**
 - A default constructor with no parameters is required.
 - It allows easy object creation.
4. **Serializable Interface**
 - Java Bean should implement `Serializable`.
 - This allows the object to be saved and transferred.

Diagram



Example

```
import java.io.Serializable;

public class StudentBean implements Serializable {
    private String name;
    private int age;

    public StudentBean() {}

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Advantages of Java Beans

1. **Reusability**
 - Beans can be reused in different applications.
 2. **Easy Maintenance**
 - Changes can be made in one place without affecting entire application.
 3. **Encapsulation**
 - Data is protected using private variables.
 4. **Platform Independent**
 - Can run on any system with JVM.
 5. **Supports MVC Architecture**
 - Helps separate data, logic, and presentation.
-

2. Explain the steps involved in developing a simple Java Bean with an example

A **Java Bean** is a reusable Java class that follows certain conventions. To develop a Java Bean, we must follow a structured approach so that it can be easily used in applications like JSP, Servlets, and GUI tools.

Steps to Develop a Simple Java Bean

Step 1: Create a Java Class

- First, create a normal Java class.
- This class will represent the Bean.

Step 2: Declare Properties as Private

- All variables (data members) must be declared as `private`.
- This ensures **encapsulation** (data hiding).

```
private String name;  
private int age;
```

Step 3: Provide Public Getter and Setter Methods

- These methods are used to **access and modify** private variables.
- Follow naming convention:
 - **Getter** → `getPropertyName()`
 - **Setter** → `setPropertyName()`

```
public String getName() {  
    return name;  
}  
  
public void setName(String name) {  
    this.name = name;  
}
```

Step 4: Add a No-Argument Constructor

- A default constructor is required.
- It allows the object to be created easily by frameworks/tools.

```
public StudentBean() {}
```

Step 5: Implement Serializable Interface

- The Bean should implement `Serializable`.
- This allows the object to be saved and transferred over a network.

```
import java.io.Serializable;
```

Complete Example of a Java Bean

```
import java.io.Serializable;

public class StudentBean implements Serializable {

    private String name;
    private int age;

    // No-argument constructor
    public StudentBean() {}

    // Getter method
    public String getName() {
        return name;
    }

    // Setter method
    public void setName(String name) {
        this.name = name;
    }

    // Getter method
    public int getAge() {
        return age;
    }

    // Setter method
    public void setAge(int age) {
        this.age = age;
    }
}
```

Working Explanation

- The class `StudentBean` stores data (name, age)
- Data is **private**, so it cannot be accessed directly
- Getter/Setter methods allow controlled access
- `Serializable` makes it transferable and storable

Diagram

Step 1: Create Class
↓
Step 2: Private Variables
↓
Step 3: Getter/Setter Methods
↓
Step 4: No-Arg Constructor
↓
Step 5: Implement Serializable
↓
Java Bean Ready

3. What are Bound Properties in Java Beans? Explain their use with an example

Definition

A **bound property** in a Java Bean is a property that **notifies other objects (listeners) whenever its value changes**.

Explanation

- In normal properties, when a value changes, **other objects are not aware** of the change.
- In **bound properties**, whenever the value changes:
 1. An **event is generated**
 2. All registered **listeners are notified**
- This is based on the **event handling mechanism** in Java.

Why are Bound Properties Used?

Bound properties are mainly used in:

- GUI applications (like Swing)
- Event-driven systems
- Applications where components need to **react to changes automatically**

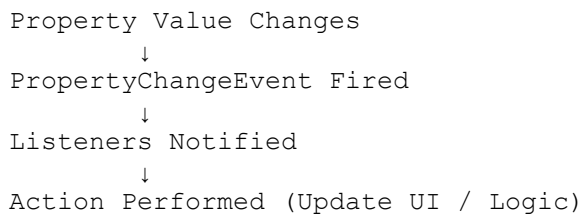
Example:

If a user changes a value in a form, other components (like labels or charts) can update automatically.

How it Works

1. A property value changes
2. A **PropertyChangeEvent** is created
3. Registered listeners are notified
4. Listeners take action

Diagram



Example of Bound Property

```
import java.beans.*;

public class StudentBean {

    private String name;

    // Support object for managing listeners
    private PropertyChangeSupport pcs = new
PropertyChangeSupport(this);

    // Add listener
    public void addPropertyChangeListener(PropertyChangeListener
listener) {
        pcs.addPropertyChangeListener(listener);
    }

    // Setter with bound property
    public void setName(String name) {
        String oldName = this.name;
        this.name = name;

        // Notify listeners about change
        pcs.firePropertyChange("name", oldName, name);
    }

    // Getter
    public String getName() {
        return name;
    }
}
```

Explanation of Example

- `PropertyChangeSupport` manages listeners
- When `setName()` is called:
 - Old value is stored
 - New value is set
 - `firePropertyChange()` notifies all listeners

Advantages

- Supports **event-driven programming**
 - Helps in **real-time updates**
 - Improves **interaction between components**
-

4. What is the BeanInfo Interface? Explain its role in customizing Java Beans

Definition

The **BeanInfo interface** (from `java.beans` package) is used to **provide additional information about a Java Bean** so that development tools (like IDEs and GUI builders) can understand and display the bean properly.

Explanation

By default, Java uses **introspection** to analyze a Bean (i.e., it automatically detects properties, methods, and events).

However, sometimes we want to:

- Hide certain properties
- Customize how properties appear
- Provide custom editors or icons

Role of BeanInfo in Customizing Java Beans

The BeanInfo interface helps in:

1. Controlling Properties

- Decide which properties are visible in tools
- Hide unnecessary properties

2. Customizing Methods

- Control which methods should be exposed
- Hide internal methods

3. Defining Events

- Specify which events are important
- Control event handling behavior

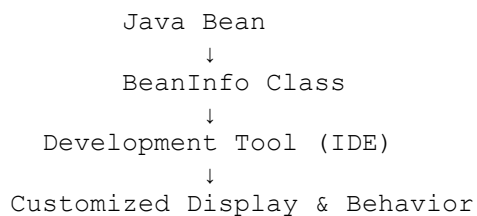
4. Providing Icons

- Custom icons can be assigned to Beans in GUI builder tools

5. Improving Tool Integration

- Helps IDEs (like NetBeans/Eclipse) display Beans properly in palettes

Diagram



How BeanInfo Works

- You create a class that implements `BeanInfo` (or extends `SimpleBeanInfo`)
- This class provides metadata about the Bean
- The IDE/tool uses this metadata instead of default introspection

Example

```
import java.beans.*;

public class StudentBeanInfo extends SimpleBeanInfo {

    public PropertyDescriptor[] getPropertyDescriptors() {
        try {
            PropertyDescriptor name =
                new PropertyDescriptor("name", StudentBean.class);

            return new PropertyDescriptor[] { name };

        } catch (IntrospectionException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Explanation of Example

- `SimpleBeanInfo` is used instead of full `BeanInfo` for simplicity
- Only the `name` property is exposed
- Other properties (if any) are hidden from tools

Advantages of BeanInfo

- Provides **full control over Bean behavior**
 - Improves **user interface in development tools**
 - Hides unnecessary details
 - Enables **custom editing support**
-

5. Explain Constrained Properties in Java Beans and Differentiate Them from Bound Properties

Definition of Constrained Properties

A **constrained property** is a property whose value **cannot be changed unless all registered listeners approve the change**.

Explanation

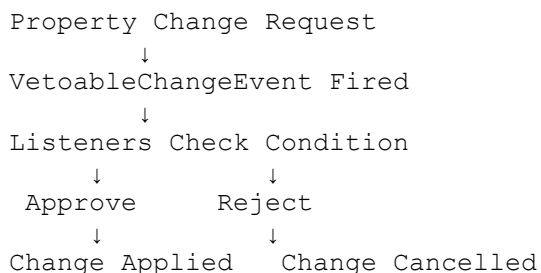
- Before updating the property value, the Bean **notifies listeners**.
- Listeners can:
 - **Allow the change**
 - **Reject (veto) the change**
- If any listener rejects the change → the value is **not updated**.

This mechanism ensures **controlled and validated updates**.

How it Works

1. A request is made to change the property
2. A **VetoableChangeEvent** is fired
3. All listeners are notified
4. If all approve → change is applied
5. If any rejects → change is cancelled

Diagram



Example of Constrained Property

```
import java.beans.*;

public class StudentBean {

    private int age;
    private VetoableChangeSupport vcs = new
VetoableChangeSupport(this);
```

```

    public void addVetoableChangeListener(VetoableChangeListener
listener) {
        vcs.addVetoableChangeListener(listener);
    }

    public void setAge(int age) throws PropertyVetoException {
        int oldAge = this.age;

        // Notify listeners before change
        vcs.fireVetoableChange("age", oldAge, age);

        // If no exception, change is accepted
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}

```

Explanation of Example

- `VetoableChangeSupport` manages listeners
- When `setAge()` is called:
 - Listeners are notified first
 - If no exception is thrown → value is updated
 - If a listener throws `PropertyVetoException` → change is rejected

Definition of Bound Properties (for comparison)

A **bound property** is a property that **notifies listeners after its value has changed**, but **cannot prevent the change**.

Difference Between Bound and Constrained Properties

Feature	Bound Property	Constrained Property
Notification Timing	After change	Before change
Can Reject Change	No	Yes
Event Type	<code>PropertyChangeEvent</code>	<code>VetoableChangeEvent</code>
Support Class	<code>PropertyChangeSupport</code>	<code>VetoableChangeSupport</code>
Use Case	Notify updates	Validate changes before applying

6. What is a Servlet? Explain the Life Cycle of a Servlet with the Methods Involved

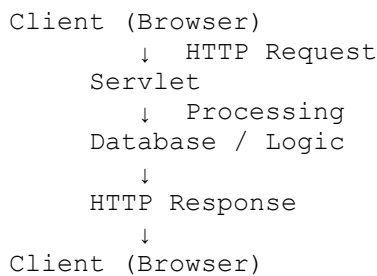
A **Servlet** is a server-side Java program that runs on a web server and is used to **handle client requests and generate dynamic responses**.

Explanation

- Servlets are used to build **web applications**.
- They act as an **intermediate layer** between:
 - Client (Web Browser)
 - Server (Database / Application logic)
- They process HTTP requests and send responses in formats like HTML.

Basic Working of a Servlet

Diagram



Servlet Life Cycle

The **life cycle of a Servlet** is managed by the web container (like Tomcat). It consists of **three main phases**:

1. Initialization Phase – `init()`

Method

```
public void init(ServletConfig config)
```

Explanation

- Called **only once** when the servlet is loaded.
- Used to **initialize resources** (like database connections).

2. Request Handling Phase – `service()`

Method

```
public void service(ServletRequest req, ServletResponse res)
```

Explanation

- Called **for every client request**.
- It decides which method to call:
 - `doGet()` → for GET request
 - `doPost()` → for POST request

3. Destruction Phase – `destroy()`

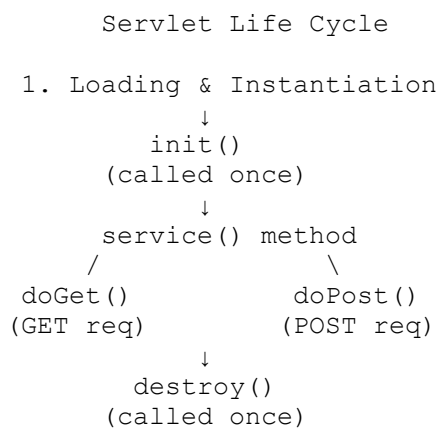
Method

```
public void destroy()
```

Explanation

- Called **once** before the servlet is removed.
- Used to **release resources** (close connections, cleanup).

Detailed Life Cycle Diagram



Step-by-Step Flow

1. Client sends a request
2. Server loads servlet (if not already loaded)
3. `init()` is called
4. For each request → `service()` is called
5. `service()` calls `doGet()` or `doPost()`
6. Response is sent to client
7. When server shuts down → `destroy()` is called

7. Write a Simple Servlet Program to Display a Message in a Web Browser

Introduction

A **Servlet** is used to generate dynamic web content. To display a message in a web browser, we create a servlet that handles an HTTP request and sends an HTML response.

Steps to Create a Simple Servlet

1. Import required packages
2. Extend `HttpServlet` class
3. Override `doGet()` method
4. Set response content type
5. Use `PrintWriter` to send output

Simple Servlet Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws IOException, ServletException {

        // Set content type
        response.setContentType("text/html");

        // Get writer object
        PrintWriter out = response.getWriter();

        // Display message in browser
        out.println("<html>");
        out.println("<body>");
        out.println("<h1>Hello, Welcome to Servlet!</h1>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

Explanation of the Program

- `HttpServlet` → Base class for HTTP servlets
- `doGet()` → Handles HTTP GET requests
- `HttpServletRequest` → Contains client request

- `HttpServletResponse` → Used to send response
- `setContentType("text/html")` → Specifies HTML output
- `PrintWriter` → Writes content to browser

Working Flow

1. User enters URL in browser
2. Request is sent to the server
3. Servlet processes the request using `doGet()`
4. HTML response is generated
5. Browser displays the message

Output in Browser

Hello, Welcome to Servlet!

8. Explain the Servlet API and the Important Classes in the `javax.servlet` Package

Definition of Servlet API

The **Servlet API** is a set of Java interfaces and classes provided by Java to develop web applications using Servlets. It defines how servlets interact with web servers and handle client requests and responses.

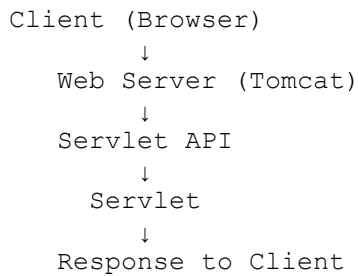
Explanation

- The Servlet API provides the **framework for creating dynamic web applications**.
- It allows communication between:
 - Web server (like Tomcat)
 - Java Servlet
 - Client (browser)

It is mainly divided into two packages:

1. `javax.servlet`
2. `javax.servlet.http`

Architecture Diagram



Important Classes and Interfaces in `javax.servlet` Package

1. Servlet (Interface)

Definition

- The root interface of all servlets.
- Defines basic lifecycle methods.

Important Methods

- `init()`
- `service()`
- `destroy()`

2. GenericServlet (Abstract Class)

Definition

- Implements the `Servlet` interface.
- Protocol-independent (can handle any request type).

Features

- Provides default implementation of lifecycle methods
- Developers override `service()` method

3. ServletRequest (Interface)

Definition

- Represents the **client request**.

Uses

- Get request parameters
- Get input stream

Example Methods

- `getParameter()`
- `getInputStream()`

4. ServletResponse (Interface)

Definition

- Represents the **response sent to client**.

Uses

- Send output
- Set content type

Example Methods

- `getWriter()`
- `setContentType()`

5. ServletConfig (Interface)

Definition

- Contains **configuration information** for a servlet.

Uses

- Access initialization parameters

6. ServletContext (Interface)

Definition

- Represents the **entire web application environment**.

Uses

- Share data between servlets
- Access server information

Hierarchy Diagram

```
Servlet (Interface)
    ↓
GenericServlet (Abstract Class)
    ↓
HttpServlet (from javax.servlet.http)
```

Important Classes in `javax.servlet.http` (Extension)

Although not asked directly, it's important:

HttpServlet

- Handles HTTP-specific requests
 - Provides methods:
 - `doGet()`
 - `doPost()`
-

9. Explain How HTTP Requests and Responses are Handled in Servlets

In web applications, communication between the client (browser) and server happens using the **HTTP protocol**. Servlets handle this communication using special objects and methods provided by the Servlet API.

Basic Concept

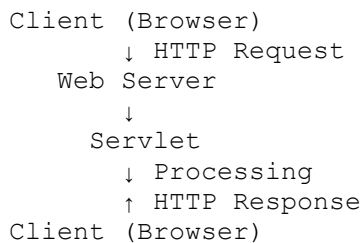
- The **client (browser)** sends an **HTTP request** to the server
- The **Servlet processes the request**
- The **Servlet sends an HTTP response** back to the client

Working Flow

Step-by-Step Process

1. User sends request (URL/form submission)
2. Web server receives the request
3. Request is forwarded to the servlet
4. Servlet processes the request
5. Response is generated
6. Response is sent back to browser

Diagram



Handling HTTP Requests

Request Object

- Represented by: `HttpServletRequest`

Purpose

- Contains all client data such as:
 - Form data
 - Query parameters
 - Headers

Common Methods

```
request.getParameter("name"); // get form data
request.getMethod();          // GET or POST
request.getHeader("User-Agent");
```

Handling HTTP Responses

Response Object

- Represented by: `HttpServletResponse`

Purpose

- Used to send data back to client

Common Methods

```
response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("<h1>Hello User</h1>");
```

Role of HttpServlet Methods

Servlet uses specific methods to handle HTTP requests:

doGet()

- Handles GET requests
- Used for retrieving data

doPost()

- Handles POST requests
- Used for sending data securely

Flow with Methods

```
Client Request
  ↓
service()
  ↓
doGet() / doPost()
  ↓
Processing
  ↓
Response Sent
```

Example

```
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws IOException {

    String name = request.getParameter("name");
```

```
response.setContentType("text/html");
PrintWriter out = response.getWriter();

out.println("<h1>Hello " + name + "</h1>");
}
```

Explanation of Example

- Client sends name parameter
- Servlet retrieves it using `request.getParameter()`
- Response is generated and sent back

Key Points

- `HttpServletRequest` → handles incoming data
 - `HttpServletResponse` → sends output
 - `doGet()` / `doPost()` → process requests
 - Servlet acts as a **controller**
-

10. What are Cookies and Sessions? Explain how they are used for session management in Servlets

HTTP is a **stateless protocol**, which means the server does not remember previous requests from the same client. To maintain user data across multiple requests, we use **Cookies and Sessions**.

Cookies

A **Cookie** is a small piece of data stored on the **client (browser)**. It is sent back to the server with every request.

Explanation

- Created by the server
- Stored in the client browser
- Used to track user information (like login details)

Example

```
Cookie cookie = new Cookie("username", "John");  
response.addCookie(cookie);
```

Diagram

Server → Sends Cookie → Client (Browser)
Client → Sends Cookie → Server (with every request)

Sessions

Definition

A **Session** is a server-side mechanism used to store user data across multiple requests.

Explanation

- Data is stored on the **server**
- Each user is identified by a **unique session ID**
- Session ID is usually stored in a cookie

Example

```
HttpSession session = request.getSession();  
session.setAttribute("username", "John");
```

Diagram

Client → Request → Server
↓
Session Created
↓
Session ID Generated
↓
Client stores Session ID (Cookie)

Session Management Using Cookies and Sessions

How it Works

Step-by-Step Flow

1. User sends request to server
2. Server creates a session
3. Server generates a **session ID**
4. Session ID is stored in a cookie
5. Cookie is sent to client
6. Client sends cookie with every request
7. Server identifies user using session ID

Combined Diagram

```
Client → Request → Server
      ↓
    Session Created
      ↓
Session ID → Stored in Cookie
      ↓
Client sends Cookie again
      ↓
Server identifies user session
```

Difference Between Cookies and Sessions

Feature	Cookies	Sessions
Storage	Client side	Server side
Security	Less secure	More secure
Size limit	Limited	Large
Performance	Faster	Slightly slower

Advantages of Session Management

- Maintains user state
- Supports login systems
- Enables personalized content
- Improves user experience

11. What are JSP Elements? Explain the Different Types of JSP Elements with Examples

JSP (Java Server Pages) is used to create **dynamic web content** by embedding Java code into HTML.

The components used inside a JSP page are called **JSP elements**.

JSP elements are special tags used in JSP to insert Java code, control page behavior, and generate dynamic content.

Types of JSP Elements

JSP elements are mainly divided into **three types**:

1. **Directive Elements**
2. **Scripting Elements**
3. **Action Elements** (often included in extended answers)

1. Directive Elements

Definition

Directives provide **instructions to the JSP container** about how the page should be processed.

Types of Directives

a) Page Directive

Used to define page-level settings.

```
<%@ page language="java" contentType="text/html" %>
```

b) Include Directive

Includes another file into JSP.

```
<%@ include file="header.jsp" %>
```


c) Taglib Directive

Used to declare custom tags.

```
<%@ taglib uri="..." prefix="c" %>
```

2. Scripting Elements

Definition

Used to insert Java code directly into JSP.

Types

a) Scriptlet

Used to write Java code.

```
<%  
int x = 10;  
out.println(x);  
%>
```

b) Expression

Used to output values.

```
<%= x %>
```

c) Declaration

Used to declare variables and methods.

```
<%! int count = 0; %>
```

3. Action Elements

Definition

Used to perform actions like including files, forwarding requests, or using JavaBeans.

Examples

a) jsp:useBean

```
<jsp:useBean id="obj" class="StudentBean" />
```

b) jsp:setProperty

```
<jsp:setProperty name="obj" property="name" value="John"/>
```

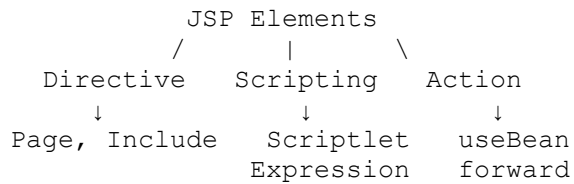
c) jsp:getProperty

```
<jsp:getProperty name="obj" property="name"/>
```

d) jsp:forward

```
<jsp:forward page="next.jsp"/>
```

Diagram



Key Points

- Directives → Control page behavior
- Scripting → Add Java code
- Actions → Perform operations

Advantages of JSP Elements

- Simplifies web development
 - Allows dynamic content generation
 - Integrates Java with HTML
 - Supports reusable components
-

12. Explain the JSP Implicit Objects and Their Uses

In JSP (Java Server Pages), developers often need objects like request, response, session, etc.

To simplify development, JSP provides **implicit objects**.

Definition

JSP implicit objects are predefined objects that are **automatically available in JSP pages** without requiring explicit declaration or initialization.

Explanation

- These objects are created by the **JSP container**
- They help in handling:
 - Client requests
 - Server responses
 - Session management
 - Output generation

List of JSP Implicit Objects and Their Uses

1. request

Type

`HttpServletRequest`

Use

- Used to get client request data
- Retrieve form parameters

Example

```
String name = request.getParameter("name");
```

2. response

Type

`HttpServletResponse`

Use

- Used to send response to client
- Set content type

Example

```
response.setContentType("text/html");
```

3. out

Type

JspWriter

Use

- Used to print output to browser

Example

```
out.println("Hello JSP");
```

4. session

Type

HttpSession

Use

- Stores user data across multiple requests

Example

```
session.setAttribute("user", "John");
```

5. application

Type

ServletContext

Use

- Used to share data across entire application

Example

```
application.setAttribute("count", 10);
```

6. config

Type

`ServletConfig`

Use

- Access servlet configuration parameters

7. pageContext

Type

`PageContext`

Use

- Provides access to all JSP scopes (page, request, session, application)

8. page

Type

`Object`

Use

- Refers to the current JSP page instance

9. exception

Type

`Throwable`

Use

- Used in error pages to handle exceptions

Advantages

- Reduces coding effort
 - Improves readability
 - Provides direct access to request/response handling
-

13. Explain How Request and Response Objects are Used in JSP

In JSP, communication between the client (browser) and server is handled using two important implicit objects:

- **request**
- **response**

These objects are provided automatically by the JSP container and are used to **receive input and send output**.

1. Request Object

Definition

The **request object** is an instance of `HttpServletRequest` used to **receive data from the client**.

Uses of Request Object

- Get form data
- Read query parameters
- Access request headers
- Store and retrieve attributes

Common Methods

```
request.getParameter("name");    // Get form data
request.getMethod();             // GET or POST
request.getAttribute("data");    // Get stored data
```

Example

```
<%
String name = request.getParameter("name");
out.println("Hello " + name);
%>
```

2. Response Object

Definition

The **response object** is an instance of `HttpServletResponse` used to **send output to the client**.

Uses of Response Object

- Send HTML response
- Set content type
- Redirect user to another page

Common Methods

```
response.setContentType("text/html");  
response.sendRedirect("home.jsp");
```

Example

```
<%  
response.setContentType("text/html");  
out.println("<h1>Welcome to JSP</h1>");  
%>
```

Working Flow in JSP

Step-by-Step Process

1. Client sends request to JSP page
2. Request object stores client data
3. JSP processes the data
4. Response object sends output
5. Browser displays the result

Combined Example

```
<%  
String name = request.getParameter("name");  
  
response.setContentType("text/html");  
  
out.println("<h1>Hello " + name + "</h1>");  
%>
```

Explanation of Example

- `request.getParameter()` → gets user input
 - `response.setContentType()` → sets output format
 - `out.println()` → displays output
-

14. What is Spring Boot? Explain its Advantages in Developing Java Applications

Spring Boot is a framework built on top of the Spring framework that is used to develop **standalone, production-ready Java applications quickly with minimal configuration.**

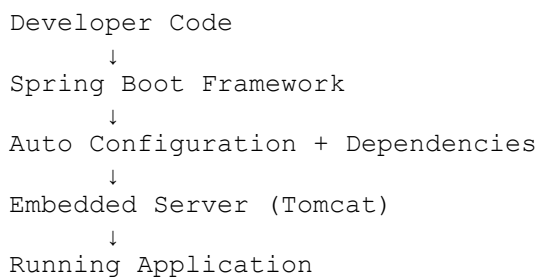
Explanation

- Spring Boot simplifies application development by:
 - Reducing configuration
 - Providing ready-to-use features
- It allows developers to focus more on **business logic** rather than setup

Key Features of Spring Boot

- Auto-configuration
- Embedded server
- Starter dependencies
- Minimal XML configuration

Architecture / Working Diagram



Advantages of Spring Boot

1. Auto Configuration

- Automatically configures the application based on dependencies
- Reduces manual setup

Example: Database connection setup is automatic

2. Standalone Applications

- No need for external servers
- Applications run using embedded servers

3. Embedded Servers

- Includes built-in servers like:
 - Tomcat
 - Jetty

No need to deploy WAR files separately

4. Starter Dependencies

- Provides pre-configured dependencies
- Simplifies project setup

Example:

```
spring-boot-starter-web
```

5. Faster Development

- Less configuration → faster coding
- Reduces boilerplate code

6. Production Ready

- Provides built-in features:
 - Monitoring
 - Metrics
 - Health checks

7. Easy Integration

- Easily integrates with:

- Databases
- REST APIs
- Microservices

8. Microservices Support

- Ideal for building microservices architecture
- Lightweight and scalable

9. Reduced XML Configuration

- Uses annotations instead of XML
- Makes code cleaner and easier to maintain

Example (Simple Spring Boot Application)

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

15. Explain the Key Components of Spring Boot and the Steps to Develop the First Spring Boot Application

Spring Boot simplifies Java application development by providing several built-in components that reduce configuration and speed up development.

Key Components of Spring Boot

1. Spring Boot Starters

Predefined dependency packages that simplify project setup.

Example

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Use

- Automatically includes required libraries
- Reduces manual dependency management

2. Auto-Configuration

Definition

Automatically configures application based on dependencies present.

Explanation

- No need to write configuration manually
- Example: Database configuration is auto-detected

3. Embedded Server

Definition

Built-in server like **Tomcat**, **Jetty**, or **Undertow**

Use

- No need to deploy WAR files
- Application runs independently

4. Spring Boot CLI (Command Line Interface)

Definition

Tool to run Spring applications quickly using commands.

5. Actuator

Definition

Provides production-ready features like:

- Monitoring

- Health checks
- Metrics

6. Spring Initializer

Definition

A tool used to create Spring Boot projects easily.

Diagram of Components

Spring Boot

↓

Starters	
Auto Configuration	
Embedded Server	
Actuator	
CLI	

Steps to Develop First Spring Boot Application

Step 1: Install Required Tools

- Java (JDK)
- IDE (Eclipse / IntelliJ)
- Maven or Gradle

Step 2: Create Project Using Spring Initializer

- Go to Spring Initializer
- Select:
 - Project type (Maven/Gradle)
 - Dependencies (Spring Web)

Step 3: Import Project into IDE

- Open project in IDE
- Build project

Step 4: Create Main Class

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Step 5: Create a Controller

```
import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

    @GetMapping("/")
    public String hello() {
        return "Hello Spring Boot!";
    }
}
```

Step 6: Run the Application

- Run main class
- Embedded server starts

Step 7: Access in Browser

<http://localhost:8080/>

16. Explain the Difference Between GenericServlet and HttpServlet with Examples

In Java Servlets, both **GenericServlet** and **HttpServlet** are important classes used to create servlets.

They are part of the **Servlet API** and help in handling client requests.

1. GenericServlet

GenericServlet is an abstract class that implements the `Servlet` interface and is **protocol-independent**.

Features

- Works with **any protocol** (not limited to HTTP)
- Provides a **generic service() method**
- Simplifies servlet development

Example of GenericServlet

```
import java.io.*;
import javax.servlet.*;

public class MyServlet extends GenericServlet {

    public void service(ServletRequest request, ServletResponse
response)
        throws IOException, ServletException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h1>Hello from GenericServlet</h1>");
    }
}
```

Explanation

- Uses `service()` method
- Handles all types of requests in one method

2. HttpServlet

Definition

HttpServlet is an abstract class that extends `GenericServlet` and is specifically designed to handle **HTTP requests**.

Features

- Supports HTTP protocol
- Provides methods:
 - `doGet()`
 - `doPost()`
 - `doPut()`
 - `doDelete()`

Example of HttpServlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class MyHttpServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h1>Hello from HttpServlet</h1>");

    }
}
```

Difference Between GenericServlet and HttpServlet

Feature	GenericServlet	HttpServlet
Protocol	Protocol-independent	HTTP-specific
Package	javax.servlet	javax.servlet.http
Methods Used	service()	doGet(), doPost(), etc.
Flexibility	More generic	More specialized
Use Case	Non-HTTP protocols	Web applications (HTTP-based)
Ease of Use	Less convenient for web	More convenient for web

17. Describe the Process of Reading Form Data in a Servlet using HttpServletRequest

In web applications, users enter data through HTML forms. This data is sent to the server, where a **Servlet** reads and processes it using the **HttpServletRequest** object.

Definition

`HttpServletRequest` is an interface that provides methods to **retrieve data sent by the client**, such as form fields, query parameters, and headers.

Process of Reading Form Data

Step-by-Step Explanation

Step 1: Create an HTML Form

- User enters data and submits the form

```
<form action="MyServlet" method="post">
  Name: <input type="text" name="username"><br>
  Age: <input type="text" name="age"><br>
  <input type="submit" value="Submit">
</form>
```

Step 2: Form Data Sent to Servlet

- Data is sent as:
 - Query string (GET)
 - Request body (POST)
-

Step 3: Servlet Receives Request

- Servlet uses `doGet()` or `doPost()` method
-

Step 4: Read Form Data using `HttpServletRequest`

Common Methods

```
request.getParameter("name");           // get single value
request.getParameterValues("name");     // get multiple values
request.getParameterNames();            // get all parameters
```

Step 5: Process Data and Send Response

Servlet Example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class MyServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {

        // Read form data
        String name = request.getParameter("username");
        String age = request.getParameter("age");

        // Send response
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h1>Name: " + name + "</h1>");
        out.println("<h1>Age: " + age + "</h1>");
    }
}
```

Working Diagram

User fills form
↓
Browser sends request (GET/POST)
↓
Servlet receives request
↓
HttpServletRequest reads data
↓
Servlet processes data
↓
Response sent to browser

Explanation

- `getParameter()` → retrieves form values
 - Values are always returned as **String**
 - Data can be processed and displayed
-

18. Explain the Role of HttpServletRequest and HttpServletResponse Objects in Handling Client-Server Communication in Servlets

In Servlet-based web applications, communication between the client (browser) and server is handled using two important objects:

- **HttpServletRequest**
- **HttpServletResponse**

These objects are provided by the Servlet container and are essential for processing requests and generating responses.

1. HttpServletRequest

Definition

`HttpServletRequest` is an interface used to **receive and process data sent by the client**.

Role

- Collects all information sent by the client
- Provides access to:
 - Form data

- Query parameters
- Headers
- Cookies

Common Methods

```
request.getParameter("name");    // Get form data
request.getMethod();             // GET or POST
request.getHeader("User-Agent");
request.getCookies();            // Get cookies
```

Example

```
String name = request.getParameter("username");
```

2. HttpServletResponse

Definition

`HttpServletResponse` is an interface used to **send data from the server to the client**.

Role

- Sends response to browser
- Controls output format
- Manages cookies and redirects

Common Methods

```
response.setContentType("text/html");
PrintWriter out = response.getWriter();
response.sendRedirect("home.jsp");
```

Example

```
out.println("<h1>Hello User</h1>");
```

Working Together (Communication Flow)

Step-by-Step Process

1. Client sends HTTP request
2. `HttpServletRequest` captures request data
3. Servlet processes the request
4. `HttpServletResponse` prepares output
5. Response is sent back to client

Diagram

```
Client (Browser)
  ↓ Request
HttpServletRequest Object
  ↓
Servlet
  ↓
HttpServletResponse Object
  ↓ Response
Client (Browser)
```

Combined Example

```
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws IOException {

    String name = request.getParameter("name");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();

    out.println("<h1>Hello " + name + "</h1>");
}
```

Key Differences

Feature HttpServletRequest HttpServletResponse

Purpose	Receive client data	Send response
Direction	Client → Server	Server → Client
Main Use	Read input	Generate output

19. Explain the Steps Required to Set Up a Spring Boot Workspace and Create a Basic Spring Boot Project

Spring Boot simplifies Java application development. Before creating applications, we need to **set up a proper workspace** with required tools and then create a basic project.

Part 1: Setting Up Spring Boot Workspace

Step 1: Install Java (JDK)

- Install **JDK (Java Development Kit)**
- Set environment variables (`JAVA_HOME`)

Step 2: Install IDE

Choose any IDE:

- Eclipse (with Spring Tools Suite plugin)
- IntelliJ IDEA

Step 3: Install Build Tool

- Maven or Gradle

Maven is most commonly used

Step 4: Verify Installation

Check in command prompt:

```
java -version  
mvn -version
```

Part 2: Creating a Spring Boot Project

Step 1: Use Spring Initializer

- Go to Spring Initializer (web or IDE)
- Provide details:
 - Project: Maven
 - Language: Java
 - Dependencies: Spring Web

Step 2: Generate and Download Project

- Click **Generate**
- Extract the project

Step 3: Import Project into IDE

- Open IDE
- Import as **Maven Project**

Step 4: Understand Project Structure

```
Project
├── src/main/java
│   └── Main Application Class
├── src/main/resources
│   └── application.properties
└── pom.xml
```

Step 5: Create Main Class

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Step 6: Create Controller

```
import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

    @GetMapping("/")
    public String hello() {
        return "Hello Spring Boot!";
    }
}
```

Step 7: Run the Application

- Run main class
- Embedded server (Tomcat) starts automatically

Step 8: Test in Browser

<http://localhost:8080/>

20. Write the Structure of a Simple Spring Boot Application and Explain the Purpose of the `@SpringBootApplication` Annotation

A Spring Boot application follows a **well-defined project structure** that makes development organized and efficient. The main class uses the `@SpringBootApplication` annotation to enable core features of Spring Boot.

Structure of a Simple Spring Boot Application

Project Structure

```
MySpringBootApplication
├── src/main/java
│   ├── com.example.demo
│   │   ├── DemoApplication.java    (Main Class)
│   │   └── Controller.java        (Controller)
├── src/main/resources
│   ├── application.properties    (Configuration file)
│   └── static / templates        (Optional files)
└── pom.xml                      (Maven dependencies)
```

Explanation of Structure

1. src/main/java

- Contains all Java source code
- Includes:
 - Main class
 - Controllers
 - Services

2. src/main/resources

- Contains configuration files
- `application.properties` is used for:
 - Server settings
 - Database configuration

3. pom.xml

- Manages dependencies using Maven
- Includes Spring Boot starter dependencies

Main Class Example

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```


Purpose of `@SpringBootApplication` Annotation

Definition

`@SpringBootApplication` is a **special annotation** that marks the main class of a Spring Boot application and enables core features.

It Combines Three Annotations

1. `@Configuration`

- Marks the class as a **configuration class**
- Used to define beans

2. `@EnableAutoConfiguration`

- Enables **automatic configuration** based on dependencies
- Reduces manual setup

3. `@ComponentScan`

- Scans the package for components like:
 - Controllers
 - Services
 - Repositories
-